词表示以及词向量

既然我们的目标是对自然语言进行处理和分析，那么就有必要回归到语言的基本要素以及本身结构。因为这门课程是国外开设的，包括一些相关的代码和论文也是国外所研究的，所以我们分析的语境主要是在**英文语境**下。而构成英语语言的基本要素就是单词，或者说是 **word** ，进而我们现在的目标就很明确了，我们需要对 **word** 进行详细的分析。

为了引入课程，我们会追问一个看起来可能很蠢的问题：**我们是如何表示词语的意思的？**你可能会说去字典里面查单词的意思呗，那更进一步地追问，字典是怎样表示一个词语的意思的？如果各位高中用过牛津字典，就会发现一个词语的解释是由**其他更简单的词语**的**指称**，这也是指称语义学所干的事。

WordNet

而当我们的目光回到**前神经网络时代**，我们的自然语言处理很大程度上依赖于上述的基本思想。所以，我们需要关注词语以及不同词语之间的关系，而在计算机中的一种常见解决方案就是——**WordNet**，这是一个显示词语关系的高级词库。其中，它存储了同义词关系和“什么是什么”这样的关系，它把英语单词按“义项”组织成同义词集，并通过上下位、反义、部分整体等关系连接起来，用来支持词义消歧、语义相似度计算、查询扩展和知识增强。

但是，这种传统方法的缺点也十分明显，具体可以概括如下：

- **词语之间的细微差别被丢失**。WordNet会告诉我们 "proficient" 和 "good" 是同义词，然而这显然只在某些语境下是正确的。
- **缺少新词语的意思。**我们的语言处在不断发展的阶段，单纯只讨论简中互联网，每隔一段时间总会诞生出一批新的词语。放到整个互联网中，新词语的数量就更多了，而 WordNet 只能依赖人工标注，这无疑是效率低的。
- **划分标准偏向于主观，没有固定标准。**因为 WordNet 是基于人类劳动所划分的词库，那么就不可避免收到主观影响。
- **需要人工手动标注和优化。**
- **不能精确地计算出词语之间的相似度。**WordNet 只能告诉你哪些词语是相似相近的，但是没有一个定量的数值指标来描述相似的程度。

正是因为 **WordNet** 存在这些无法克服的问题，所以我们必须开始思考其他表示词义的具体方法。词语作为一种离散化的符号，为了在计算机中表示和计算，那么我们显然要对**词语**进行**代数化**。

One-Hot编码

假设我们有一个词库，其中的词语可以被用一种叫做 **One-Hot** 向量进行表示，比如我们有一个包含6个词的词库，那么前两个词语可以被表述如下：

$$word_1 = (1, 0, 0, 0, 0, 0)$$
$$word_2 = (0, 1, 0, 0, 0, 0)$$

后面的词语以此类推，这是一种很容易想到的代数化方法，但是新的问题也随之出现了。

> **Tip**
>
> **One-Hot** 之所以叫这个名称，是因为对应词语向量的分量中只有一个是 "1"，其他的分量都是 "0"

最显著的问题就是，词库中的词语数量决定了我们向量的维度，而我们常用的词汇怎么也得几十万了，一个几十万维度的向量用以相关计算这是不现实的。除去这个计算方面上的问题，这种代数化方法暴力地只是对词语进行了分类，但是也抛弃了词语之间所有的关联信息。因为如果你考虑两个向量的**点积**，结果为零，代表着这两个向量正交，也就是没有任何关联。

> **Note**
>
> 这种表示方法也被称为 **局部表示**，意味着事物只在某一个点上被表示。

词向量

所以，以上两种方法都存在着一定程度的问题，我们需要进一步设计方法。现在，让我们再次回顾一下语言学中表示一个词的方法，除了使用相近的词来标注它，我们还可以通过上下文的语境来表示理解它。更进一步地，我们可以说一个词的意思是由离它最近的一些词语所决定的，基于此我们引入了**词向量**的概念，也是现代统计自然语言处理中最成功的理念之一。

所以，现在如果我要想了解一个词的意思，我会说给我一些包含这个词的上下文，然后用一个短的密集向量来表示，比如：

$$\text{banking} = (0.286, 0.792, -0.177, -0.107, 0.109, -0.542, 0.349, 0.271)^T$$

这样构造出来的词向量的性质就会非常好，向量的维度不会很大，不同的词之间的关联性也被保留。换句话说，我们是将每个词汇嵌入到一个高维的向量空间中。基于此，我们需要学习系统生成词向量的具体算法——**Word2vec** 。

利用这个方式计算词向量的过程大致如下：

1. 我们有一段文字，在 **NLP** 中称其为语料库，也可以看作是由词语组成的一个列表。
2. 先初始化语料中的每个单词的向量表示，相当于给定初值条件。
3. 遍历语料中的每个位置 t ，中间为中心词 c ，其附近存在着外部词 o 。
4. 利用 c 和 o 的词向量相似度来计算对于给定 c 的 o 的概率。
5. 不断优化词向量的数值，使得上一步得到计算的概率最大化。

Word2vec 数学推导过程

看起来可能比较绕，但起码我们能看出来，词向量的生成过程是一个优化迭代的过程。下面，我们结合一个例子来讲解一下，讲解过程中也会涉及到相关的数学推导，下面是一个英文句子的片段：

$$\cdots \ problems\ turning\ into\ banking\ crises\ as\ \cdots$$

考虑**中心词为 into，左右长度为 2 的区间**，将中心词记为 w_t ，将语句看作词构成的列表。那么所有外部词的出现概率实际上是在中心词出现条件下的概率，也就是计算条件概率，并且是一个联合条件概率。那么显然左右长度为 2 的区间内外部词的出现概率为：

$$P(w_{t-2}, w_{t-1}, w_{t+1}, w_{t+2} \mid w_t)$$

将其推广一般性而言，对于一个左右长度为 m 的区间，其中对应外部词的出现概率为：

$$P(w_{t-m}, \cdots, w_{t+j}, \cdots, w_{t+m} \mid w_t) \quad j \in [-m, m] \text{ 且 } j \neq 0$$

这个式子让我们非常熟悉啊，可以想一下机器学习中的朴素贝叶斯分类，那里我们做了一个很巧妙的化简，使得其能够更方便地进行计算。也就是，我们做一个朴素的假设：我们认为在给定中心词 w_t 后，外部词的出现**彼此相互独立**。那么，联合条件概率就可以被化简为一系列简单概率乘积，为了严谨我们采用约等：

$$P(w_{t-m}, \cdots, w_{t+j}, \cdots, w_{t+m} \mid w_t) \approx \prod_{j=-m}^{m} P(w_{t+j} \mid w_t) \quad j \neq 0$$

因为对应的上下文词 (w_{t+j}, w_t) 已经在语料中出现，所以我们认为这个词对的出现概率是比较大的。为了方便处理，我们的目标直接设定为最大化上述式子的结果。对于这样的一个联合条件概率，其真实分布我们不知道，所以我们需要使用到概率统计中的**参数估计**。为此，我们引入一个参数，设为 θ，那么继续做如下推导。

加入参数后，我们整段语料的**似然函数**如下：

$$L(\theta) = \prod_{t=1}^{T} \prod_{j=-m}^{m} P(w_{t+j} \mid w_t; \theta) \quad j \neq 0$$

也就是目标在于找到合适的参数，使其似然函数值最大，也就是**极大似然估计**。根据概率论知识，我们知道最大化似然等价于最大化对数似然，同时取对数后可以将不好处理的乘积转化为求和形式。

$$\log L(\theta) = \sum_{t=1}^{T} \sum_{j=-m}^{m} \log P(w_{t+j} \mid w_t; \theta) \quad j \neq 0$$

在对数似然函数上我们进一步转化，得到**目标函数（有时也可叫做损失函数）**：

$$J(\theta) = -\frac{1}{T} \log L(\theta) = -\frac{1}{T} \sum_{t=1}^{T} \sum_{j=-m}^{m} \log P(w_{t+j} \mid w_t; \theta) \quad j \neq 0$$

现在，经过一通分析后，我们的目标就很简单了，优化目标函数，使其取得最小值（因为乘了个负号）。那么新的问题也来了，我们如何计算单项的条件概率 $P(w_{t+j} \mid w_t; \theta)$，解决方法如下：

这个时候我们需要用到了对应词的词向量了，前面我初始化了不同词的词向量，这里我们设定每个词都有两个词向量，分别为 u, v ，表示这个词在作为上下文和中心词时的词向量，可以证明最后两个向量会被迭代到接近。

而我们的目标就是得到优化后的词向量，所以我们的参数 θ 在这里实际上是一个包含了 $2T$ 个向量的矩阵。并且这个矩阵由两部分组成，一部分是由 T 个 u_i 组成的上下文矩阵 U ，另一部分是由 T 个 v_i 组成的上下文矩阵 V ，其中 u_i, v_i 是第 i 个词在作为上下文和中心词时的词向量。

现在考虑对于给定的中心词 w_t 和上下文词 w_{t+j}，我们使用 **softmax函数** 计算对应条件概率，具体如下：

$$P(w_{t+j} \mid w_t; \theta) = \frac{\exp(u_{t+j}^T v_t)}{\sum_{i=1}^{T} \exp(u_i^T v_t)}$$

> **Note**
>
> 对于这个式子的理解，我是这样想的。因为我们有了初始化的词向量，然后很自然地可以使用向量的点积来衡量相似程度。但是向量的点积结果是一个实数，我们希望得到的是一个位于 0，1 之间的概率值，所以我们考虑使用机器学习中常使用的 **softmax** 处理。

所以，目标函数中的通项可以改写为：

$$J_{t,j}(\theta) = -\log P(w_{t+j} \mid w_t; \theta) = -u_{t+j}^T v_t + \log \sum_{i=1}^{T} \exp(u_i^T v_t)$$

由于原式是对通项的求和，所以进一步地，目标函数可以改写为：

$$J(\theta) = \frac{1}{T} \sum_{t=1}^{T} \sum_{j=-m}^{m} J_{t,j}(\theta) \quad j \neq 0$$

现在我们可以开始求解这个问题了，要找到最优参数，本质上就是最小化目标函数，既然要对一个多元函数做优化，显然我们要求梯度，而梯度是线性可加的。所以我们先看单个样本损失的梯度，也就是 $J_{t,j}(\theta)$ 。

首先，对 v_t 求偏导如下，令 $Z = \sum_{i=1}^{T} \exp(u_i^T v_t)$ ，

$$\frac{\partial J_{t,j}}{\partial v_t} = -u_{t+j} + \frac{1}{Z} \frac{\partial Z}{\partial v_t}$$

而

$$\frac{\partial Z}{\partial v_t} = \sum_{i=1}^{T} \exp(u_i^T v_t) u_i$$

所以，

$$\frac{\partial J_{t,j}}{\partial v_t} = -u_{t+j} + \sum_{i=1}^{T} \frac{\exp(u_i^T v_t)}{Z} u_i$$

注意到

$$\frac{\exp(u_i^T v_t)}{Z} = P(w_i \mid w_t; \theta)$$

因此：

$$\frac{\partial J_{t,j}}{\partial v_t} = \sum_{i=1}^{T} P(w_i \mid w_t; \theta) u_i - u_{t+j}$$

其次，我们是对 u_i 求偏导，这里会分为两种情况：

$$\frac{\partial J_{t,j}}{\partial u_i} = \begin{cases} \frac{1}{Z}\exp(u_i^T v_t) v_t - v_t = (P(w_i \mid w_t; \theta) - 1) v_t & t+j=i \\ \frac{1}{Z}\exp(u_i^T v_t) v_t = P(w_i \mid w_t; \theta) v_t & t+j \neq i \end{cases}$$

求出样本的梯度后，总损失函数的梯度也就随之确定了，之后便可以使用梯度下降的方法不断优化参数，最后就能得到词库中所有词语的词向量了。梯度下降的过程大致如下，我们优化的参数为 θ ，那么迭代过程如下：

$$\theta_{new} = \theta_{old} - \eta \nabla J(\theta)$$

> **Note**
>
> 上面的式子是我们正常的梯度下降方法，但是在实际运用过程中，这种方法存在着一些问题。前文中我们提到了，θ 是词向量构成的矩阵，如果我们的词库比较大时，矩阵的维数将会十分巨大。这个时候，我们单次迭代计算梯度的时间就会很长，进而使总的训练时长变得很长，这是不利于我们的模型训练的。为此，我们引入了**随机梯度下降**的方法，即每次迭代计算梯度时，我们只选取一些样本，计算这些样本的梯度将其作为总体的梯度。换句话说，样本的梯度可以看作是在准确的总体梯度上引入了**噪声**，而在神经网络中有一个很有意思的现象，对参数加入一定的噪声，模型的效果会更好。

一些存在的问题

现在，让我们先回答上一部分遗留下来的一个问题：为什么我们要给每个词设定两个向量，分别作为中心词和外部词时的向量。可以这样理解，两个向量都会在不断迭代的过程中被优化，它们也代表了不同角度的学习过程。

- 当作为中心词时，我们最后训练得到的结果是使用这个词去预测其他词的时候，这个词在 n 维空间内的表示。
- 当作为外部词时，我们最后训练得到的结果是这个词被其他词预测的时候，这个词在 n 维空间内的表示。

所以两个向量只是从不同的角度学习得到的词向量，如果学习的算法正常进行，最后得到的两个向量将会接近，这时可以对其取均值作为对应词的词向量或者是简单的取其中之一即可。

现在而言，我们可以说是得到了词库中的每个词的词向量表示，我们还需要对已经生成的词向量进行**质量评估**，评估大致有两种方式：

- **内部评估**：词向量本质上是一个向量，那么也可以进行向量之间的运算。所以，我们考虑这两个词 "man" 和 "king" ，那么将这两个词向量相减得到一个向量。这个向量在一定程度上代表了两个词之间的关系，我将这个向量加到 "woman" 对应的词向量上去，那么距离结果最近的词向量很大可能就是 "queen" 。这是在理想状况下的推导，所以我们可以利用这个来评估我们生成的词向量的质量。
- **外部评估**：我们可以将我们的词向量放在真实情景下去做一个检验，先让人类来给词语之间的相似程度随机打分，将不同人的打分结果进行汇总，得到真实情景下的词语关系。再将词向量的训练结果与人类的打分进行比对，从而判断词向量的质量。

现在我们还面临着一个问题，一个词并不只有一种意思，也就是存在着一词多义现象。这个时候，单个词向量不能够很好地表示词的所有意思。针对于这个问题，主要有以下几种的处理方式：

1. **接受这个缺陷**，也就是说，不专门处理多义性，默认一个词一个向量。在大多数的任务里，这种方法足够应用了，因为高频语义往往占据主导地位。但是，当我们的任务需要**精细语义划分**时，这种方法会出问题。
2. **多原型词向量**，一个词不只对应一个向量，而是对应多个义项向量。训练时先根据上下文，把同一个词出现的不同语境聚类，然后为每个聚类学习一个向量。这样在句子里遇到多义词时，就根据上下文选择最合适的那个义项向量。
3. 现代 NLP 里，更主流的方法不是给词固定多个向量，而是：**同一个词在不同上下文里，动态生成不同向量**。